Mahfuz Chowdhury

+44 7392 434467 | London, UK | mahfuzcswe@gmail.com | linkedin.com/in/mahfuzcswe | github.com/mc-161

EDUCATION

City, University of London | **BSc Computer Science, First Class Honours** 2022 – 2026
Modules: Algorithms, Data Structures, Databases, Operating Systems, Networking, Systems Architecture, Cloud Computing, Discrete Maths, NLP

EXPERIENCE

American Express Jun 2025 – Aug 2025
Software Engineer Intern · Java, Spring Boot, Go, PostgreSQL, Redis, Kafka, Nginx, Kubernetes *Burgess Hill, Sussex*

- **Built Go services into a live transaction processor** during its Java/Spring Boot-to-Go migration, keeping **5 downstream consumers** running by adding versioned contracts on **12 REST/SOAP endpoints**.
- **Gave code reviews on teammates' pull requests** and **verified every AI-generated test** before merge: scaffolded integration tests with GitHub Copilot Enterprise, then rewrote the assertions that missed real cases.
- **Co-built the team's developer hub and event-tracing app** with an intern I onboarded, shaped by Scrum and product-owner feedback; **adopted by 10 of 12 engineering teams**. Tracing any partner transaction and its sub-events from one ID cut diagnosis from **10–20 min to seconds** (product owners had waited up to a day).
- **Gave 3 product-owner teams self-serve payment data** via a Kafka consumer (**10,000+ events/day**) and React dashboard.
- **Cut sub-event reads from 5 queries per transaction to 2** in the payment processor via SQL joins and Redis hot lookups.
- **Debugged and fixed a reverse-proxy failure blocking the tracing app** in staging via environment-aware Nginx and Go routing.
- **Presented the migration and observability work to an SVP and 3 directors** (UK/US); ran feedback sessions with 5 Scrum teams.

DotPlot / Tech Academia Jul 2024 – Aug 2024
Software Engineering Pathway · React, TypeScript, Node.js, Express, MongoDB, JWT, Git, CI/CD *Remote*

- **Won 1st place of 7 teams** leading a team of 4 to build Sonomon, a full-stack patient-management and breast-scan visualisation app, deployed 24 hours early; cleared JWT auth and 3D scan-rendering blockers through pair programming.
- Cut page load time from **6.9s to 1.9s** (~72% faster) by lazy-loading scans and 3D models in the React frontend.

OPEN SOURCE

formancehq/ledger | Programmable Double-Entry Financial Ledger · *Go, PostgreSQL* Jun 2026 – Present

- Cut chart-of-accounts pattern matching **~9x (2,580ns → 287ns, 62 → 4 allocs/op)** by replacing per-posting regex recompilation with a cache; verified with benchstat across 6 runs.
- **Redesigned the change after maintainer code review** to keep exact JSON round-trip equality under the project's no-behaviour-change policy; bounded the cache with the project's existing cache library to close an unbounded memory-growth risk; added a regression test.

PROJECTS

Price Watch | Distributed Price Monitoring & Alerting · *Java, Spring Boot, PostgreSQL, Testcontainers* Aug 2026 – Sep 2026

- Stopped concurrent scheduler instances double-claiming work: reproduced the race at a weaker isolation level, fixed it with **SELECT FOR UPDATE SKIP LOCKED** and verified it on real PostgreSQL via Testcontainers.
- Prevented duplicate or lost alerts across crashes by pairing a **transactional outbox with idempotent dispatch**; kept polling fault-tolerant under rate limits with per-source adaptive backoff and circuit breakers.

Risk Watch | Portfolio Risk Platform (Final Year) · *C++17, Python, FastAPI, Go, Redis, TimescaleDB, React* Sep 2025 – Apr 2026

- Built a **C++17 quant risk engine** behind pybind11, running **10,000-path Monte Carlo in 0.41s**; 50+ metrics verified against NumPy/SciPy to 1e-6.
- Moved Monte Carlo and backtest jobs onto Celery workers, pushing completion events via per-user Redis Streams to a rate-limited Go WebSocket gateway so the API never blocks.
- **Deployed and supported in production** on Northflank and Vercel with per-service monitoring and GitHub Actions CI/CD; **fixed a production job-queue failure** (Redis tier limits made workers skip jobs) by adding a fallback data source, migrating Redis and merging triggers.

Trading Floor | Real-Time Multiplayer Exchange · *Go, WebSocket, TypeScript* Sep 2026 – Present

- Real-time exchange where multiple users trade live against one **shared order book**; a Go price–time priority matching engine sequences concurrent orders by server arrival.
- **Event-sourced** design gives byte-identical replay and reconnect recovery, so any session can be audited and rebuilt exactly.

Powerlifting Analytics Engine | In-Memory Columnar Engine · *C++20, Go, gRPC, Google Benchmark* Jul 2026 – Aug 2026

- Built a columnar engine over **4M results**: ranking queries in **5.2 ms vs 103 ms** row-oriented (~20×) at half the memory; normalised **405 weight-class labels** across federations into one consistent scheme.
- Ran the engine as a standalone **gRPC service** behind the Go API, not cgo, so engine failures degrade one endpoint, not the API; verified results against an independent implementation.

SKILLS & INTERESTS

- **Languages:** Java, Go, Python, SQL, TypeScript, JavaScript, C++, C, Bash
- **Frameworks & Tools:** Spring Boot, JUnit, Testcontainers, FastAPI, Celery, React, gRPC, pybind11, GitHub Copilot, Claude Code
- **Data & Messaging:** PostgreSQL, TimescaleDB, Redis, Kafka, MongoDB
- **Cloud & Infrastructure:** Docker, Kubernetes, AWS, CI/CD (GitHub Actions), Nginx, Linux/Unix, Git, Maven, CMake
- **Activities:** Powerlifting (university meets); running (parkrun); basketball; badminton